

Introduction to Object-Oriented Programming in Java

Chapter 1: Classes, Objects, and Constructors

The Class is defined as a blueprint or template from which individual objects are instantiated, encapsulating both data fields (state) and methods (behavior).

An Object represents a concrete instance of a class occupying memory in the JVM heap, maintaining its own discrete state through instance variables.

The Constructor is defined as a specialized initialization subroutine invoked automatically during object instantiation via the new keyword to initialize object state.

The This Keyword denotes a reference variable in Java that refers directly to the current executing object instance, resolving naming collisions between instance variables and parameters.

Chapter 2: Encapsulation and Information Hiding

Encapsulation refers to the core OOP principle of bundling data and the methods that mutate that data into a single cohesive unit, restricting direct external access to internal state.

Access Modifiers determine the scope and visibility of class members across packages and subclasses. The private modifier enforces strict information hiding, accessible solely within the defining class.

A Getter Method serves as a read-only accessor providing controlled observation of private fields without permitting arbitrary external modification.

A Setter Method enables safe mutation of encapsulated properties, providing an enforcement boundary where validation rules and invariants can be asserted.

Chapter 3: Inheritance and Class Hierarchies

Inheritance represents a fundamental mechanism where a derived subclass inherits fields and public behaviors from an existing superclass using the extends keyword.

The Super Keyword represents a reference to the immediate parent class, enabling child classes to invoke overridden methods or delegate constructor chaining up the inheritance hierarchy.

Method Overriding enables a subclass to provide a specific, customized implementation of a method already declared in its superclass, marked by the @Override annotation.

Method Overloading defines the ability to provide multiple methods in the same class sharing the identical name but differing in parameter types, count, or signature.

Chapter 4: Polymorphism and Dynamic Dispatch

Polymorphism refers to the object-oriented capability allowing entities of differing concrete types to be handled through a single uniform superclass or interface reference.

Dynamic Method Dispatch represents the runtime mechanism in the Java Virtual Machine that resolves calls to overridden methods dynamically at execution time based on the actual object instance rather than the reference variable type.

Upcasting enables treating a specific child instance as its more general superclass reference, guaranteeing type safety without explicit casting.

Downcasting allows re-converting a generalized superclass reference back into a specific subclass type, verified at runtime via the instanceof operator to prevent ClassCastException.

Chapter 5: Abstraction, Interfaces, and SOLID Design

An Abstract Class serves as an incomplete class template that cannot be directly instantiated, containing one or more unimplemented abstract methods intended for subclass specialization.

An Interface defines a formal contract of behaviors that implementing classes must fulfill using the implements keyword, facilitating multiple interface inheritance and loose coupling.

Composition denotes an architectural relationship where a complex object contains instances of other classes ("has-a" relationship), preferred over rigid inheritance ("is-a" relationship) for flexible system design.

The Single Responsibility Principle asserts that a class should encapsulate only a single concern and possess exactly one cohesive reason to change.